Gate L1 — i18n Framework

Status: Spec (PM, architect-revised 2026-04-29) **Target:** May 8, 2026 (architect estimate: ~8–8.5 working days from May 1 start, ~1 week slack) **Branch (when cut):** `gate/L1-i18n-framework` **Spec branches:** `docs/i18n-planning` (original), `docs/i18n-architecture` (revisions A1–A4, B1–B2) **Inputs:** `docs/planning/i18n/README.md` (D1–D18), `docs/planning/architecture/i18n-architecture.md`

L1 establishes the multilingual plumbing. After L1, the public site looks identical to today (English only, no visible locale switcher unless `NEXT_PUBLIC_FEATURE_I18N=1`), but every layer needed to ship French in L2 is in place: routing, middleware, translatable model fields, message catalogs extracted from the source, fonts widened, and a working locale switcher behind the flag.

No translations are produced in L1. L1 produces empty French/German/Spanish message catalogs and empty `_fr` / `_de` / `_es` fields. L2 is the first gate that translates anything.

> ***Architecture revisions (2026-04-29).*** *The architecture review (`docs/planning/architecture/i18n-architecture.md` §11) added four items to L1 (A1–A4) and clarified two L3 deferrals (B1–B2). Inline edits below carry the story-level updates; §A1–A4 / §B1–B2 below summarizes the net deltas. The story acceptance criteria are authoritative; the architecture doc is the technical reference.*

Stop-ship checkpoints

L1 has two natural checkpoints. Either is shippable on its own.

- **Checkpoint A — Backend ready.** Django middleware, modeltranslation, migrations, admin can edit `_fr` / `_de` / `_es` columns. Frontend untouched.
- **Checkpoint B — Frontend ready.** `next-intl` wired, message catalogs extracted, locale switcher built, fonts widened, all routes work in `/fr/`, `/de/`, `/es/` (rendering English fallback content).
- **Full L1 — Both layers integrated.** Frontend reads `Accept-Language` → backend returns the right language → fallback to English on missing content. End-to-end smoke test in all four locales passes.

---

1. User stories

Backend foundation

S1 — Django i18n middleware and language config

Audience: All users (invisible until content is translated). **Checkpoint:** A. **Scope:** Backend. **Complexity:** S.

As a platform operator, I want Django's locale middleware enabled and the four supported languages declared, so that subsequent translation work has the correct foundation.

- Add `django.middleware.locale.LocaleMiddleware` to `MIDDLEWARE` in `backend/config/settings/base.py`, immediately after `SessionMiddleware` and before `CommonMiddleware`.
- Set `LANGUAGES = [("en", "English"), ("fr", "Français"), ("de", "Deutsch"), ("es", "Español")]`.
- Keep `LANGUAGE_CODE = "en-us"` (default fallback).
- `USE_I18N = True` is already set; confirm.
- Add `LOCALE_PATHS = [BASE_DIR / "locale"]` and create the `backend/locale/` directory with a `.gitkeep`.

- No model changes in this story.

Acceptance:

- `manage.py compilemessages` runs cleanly (against empty catalogs).
- A request with `Accept-Language: fr` returns `request.LANGUAGE_CODE == "fr"`. Verified via a one-line debug view or unit test that's removed before merge.

S2 — `django-modeltranslation` installed and configured

Audience: Admin users (Tier 5). **Checkpoint:** A. **Scope:** Backend. **Complexity:** M.

As a content reviewer, I want translatable model fields available in Django admin so that I can edit French / German / Spanish copy in the side-by-side workflow when L3 ships.

- Add `modeltranslation` to `INSTALLED_APPS` (above `django.contrib.admin` per the library's docs).
- Add `MODELTRANSLATION_DEFAULT_LANGUAGE = "en"`, `MODELTRANSLATION_LANGUAGES = ("en", "fr", "de", "es")`, `MODELTRANSLATION_FALLBACK_LANGUAGES = {"default": ("en",)}`.
- Create `backend/species/translation.py` registering the initial translatable fields:
  - `Species`: `description`, `ecology_notes`, `distribution_narrative`, `morphology`.
  - `Taxon`: `common_family_name`.
- Run `makemigrations` and review the generated migration. Confirm new columns are nullable and that existing English content is preserved on the original column (the `_en` suffix aliasing approach — see `README.md` open-question 4 for the architect's call).
- Update `backend/species/admin.py` (or wherever Species is registered) so the admin form shows tabs / fieldsets for all four locales per translatable field. Modeltranslation provides `TranslationAdmin` for this — use it.

Acceptance:

- Migrations apply cleanly on a fresh database AND on a database with existing species rows.
- After migration, an existing species's English `description` is still readable through the admin and through the API (no regression).
- The admin form for Species shows four-locale tabs / inline editing for every registered field.
- A test populates `description_fr` for one species, requests the species detail with `Accept-Language: fr`, and gets the French content. Same request with `Accept-Language: de` falls back to English.

S3 — DRF serializers respect active language

Audience: All API consumers (frontend, third-party, future GBIF export). **Checkpoint:** A. **Scope:** Backend. **Complexity:** S.

As an API consumer, I want endpoints to return localized content based on `Accept-Language`, so that the frontend can request `/api/v1/species/123/` with `Accept-Language: fr` and get French content.

- Verify that `LocaleMiddleware` from S1 is correctly setting the active language for DRF requests (it should — DRF runs through Django middleware).
- Translatable fields on Species, Taxon, etc. return the active-language value automatically via modeltranslation's `LANGUAGE_CODE`-aware getters. No serializer changes required for the read path *if* modeltranslation is configured correctly.

- Add a fallback indicator: when a translatable field falls back to English because the requested locale's value is empty or below `human_approved` (L3 work), the serializer adds a sibling `<field>_locale_actual` key with the locale that was actually returned. Frontend uses this to render the "(English)" badge per D6.
- Implement as a `TranslationActualLocaleMixin` on the relevant serializers (architecture doc §4 has the implementation pattern). In L1 the mixin checks "column populated/empty" only; the gating against `TranslationStatus.status == human_approved` is L3, controlled by an `I18N_ENFORCE_REVIEW_GATE` setting that is `False` in L1, `True` in L3.
- Note: in L1, no field has any non-English content yet, so the fallback indicator will always read `"en"`. The wiring is what L1 delivers; the data follows in L2/L3.
- **A2 — `apiFetch` Accept-Language threading (architect addition).** The frontend `apiFetch` helper (`frontend/lib/api.ts`) must accept an optional `locale` parameter and forward it as `Accept-Language` on every request. The locale comes from `next-intl`'s server-side `getLocale()` call inside the page that invokes the fetcher. Without this, Django can't serve locale-aware content even when modeltranslation is wired correctly (see architecture doc §9 R6). Also update the small set of callers that originate locale-bearing requests so they thread the locale through.

Acceptance:

- `GET /api/v1/species/123/` with `Accept-Language: fr` returns the same payload shape as today plus `*_locale_actual` keys reading `"en"` (because no FR content exists yet).
- `?lang=fr` query parameter overrides `Accept-Language`. Same payload result.
- `apiFetch({ locale: "fr" })` sets `Accept-Language: fr` on the outgoing request; an explicit `Accept-Language` in `headers` wins (escape hatch).
- The page-level callers under `frontend/app/[locale]/...` pass `getLocale()` into their `apiFetch` calls.
- Existing API contract tests still pass — no breaking field changes (the `*_locale_actual` keys are JSON-additive; OpenAPI schema regen via `npm run gen:types` updates the committed `lib/api-types.ts` in the same PR).

S4 — Translation status model (foundation for L3 review UI)

Audience: Admin reviewers (Alex, colleagues at Tier 5). **Checkpoint:** A. **Scope:** Backend. **Complexity:** M.

As a reviewer, I want every translatable field/locale combo to track its review state so that the L3 admin UI can filter by `mt_draft`, `writer_reviewed`, `human_approved`.

- Create a `TranslationStatus` model in `backend/i18n/` (new app):
  - `content_type` (FK to `ContentType`)
  - `object_id` (positive integer)
  - `field` (CharField, max 100)
  - `locale` (CharField, choices = LANGUAGES minus EN — EN is always the source)
  - `status` (CharField, choices: `mt_draft`, `writer_reviewed`, `human_approved`, `published`; default `mt_draft`)
  - `mt_provider` (CharField, default `"deepl"`, nullable)
  - `mt_translated_at` (DateTimeField, nullable)
  - `writer_reviewed_at` (DateTimeField, nullable)
  - `human_approved_at` (DateTimeField, nullable)
  - `human_approved_by` (FK to User, nullable, on_delete=SET_NULL)
  - `notes` (TextField, blank)
  - Unique together: `(content_type, object_id, field, locale)`.

    - `__str__` returns a human-readable label.
- Register in admin with read-only mode for L1. The L3 spec wires the side-by-side editor.
- **B1 — Signal handlers explicitly excluded from L1 (architect deferral).** Auto-creating / auto-invalidating `TranslationStatus` rows in response to translatable-field saves is L3 work, not L1. In L1, rows can only be created/edited manually through admin; this is acceptable because no `_fr` / `_de` / `_es` columns receive writes in L1 (catalogs are byte-identical English placeholders, no MT pipeline yet). See architecture doc §5 for the L3 signal design.
- Add the recommended indexes from architecture doc §5: composite index on `(content_type, object_id)` plus a covering index on `(locale, status)` for the L3 admin filter UI.
- No frontend exposure in L1.

Acceptance:

- Migration applies cleanly.
- Creating a `TranslationStatus` row through admin works.
- Unique constraint prevents duplicates.
- A unit test creates rows for one species + one field + three locales and queries them.
- The two indexes are present and used by `EXPLAIN` for the typical L3 query patterns (`WHERE status = 'mt_draft' AND locale = 'fr'`).

Frontend foundation

S5 — `next-intl` installed and middleware wired

Audience: All visitors (invisible until S10 flips the flag). **Checkpoint:** B. **Scope:** Frontend. **Complexity:** M.

As a visitor, I want the site to recognize URL locale prefixes (`/fr/` , `/de/` , `/es/`) so that I can navigate to localized routes once L2 ships content.

- Install `next-intl@^3` in `frontend/package.json` .
- Add `frontend/i18n.ts` (or `frontend/lib/i18n.ts`) with `getRequestConfig` returning the active locale's messages.
- Add `frontend/middleware.ts` integration: `next-intl` middleware composed with the existing auth middleware (do not replace it). Locale prefix mode: `as-needed` . Default locale: `en` . Locales list: `[en, fr, de, es]` .
- **A3 — Composed middleware refactor (architect addition).** Implement the composition pattern from architecture doc §9 R1: `next-intl` middleware runs first to handle locale negotiation; the existing auth gate runs second against the (post-rewrite) path. Refactor the existing logic in `frontend/middleware.ts` into an `authGate(req)` helper called from the composed entry point. Update the matcher to accept optional locale segments: `"/((?:fr|de|es)/)?account/:path*"` and `"/((?:fr|de|es)/)?dashboard/coordinator/:path*"` .
- **A3 — Locale-aware redirects.** Update `redirectToLogin` (currently `frontend/middleware.ts`) and the auth-flag-off `/` fallback to construct locale-aware paths: `/fr/login?callbackUrl=/fr/account` instead of `/login?callbackUrl=/account` when the original path has a locale prefix. The `callbackUrl` must preserve the locale.
- Restructure `frontend/app/` to nest under `frontend/app/[locale]/...` per the next-intl App Router guide. The non-prefixed root `/` continues to resolve to the English locale through middleware rewriting; existing URLs like `/species/123` are not broken. `globals.css` stays at `frontend/app/globals.css` ; `layout.tsx` becomes the `[locale]` layout while a new minimal root layout handles `<html>` .
- Set `<html lang={locale}>` in the root layout dynamically.
- No translated content yet — every locale renders the English message catalog (S6).

Acceptance:

- `/species/123` works (English).
- `/fr/species/123` works and renders the same English content (because no FR translations exist) but `<html lang>` reads `fr`.
- `/de/species/123`, `/es/species/123` likewise.
- Existing tests continue to pass after the route restructure.
- **Eight new middleware test cases in `frontend/middleware.test.ts`** covering locale × auth scenarios from architecture doc §9 R1: `/fr/account` anonymous redirects to `/fr/login?callbackUrl=/fr/account`; `/de/dashboard/coordinator` Tier 2 redirects to `/de/login?callbackUrl=/de/dashboard/coordinator`; `/es/account` Tier 1 with flag on passes through; etc. Without these, the auth gate breaks silently in production under locale prefixes.

S6 — Message catalogs scaffolded with EN baseline

Audience: Translators and reviewers (workflow-internal). **Checkpoint:** B. **Scope:** Frontend. **Complexity:** L.

As a developer, I want every hardcoded UI string moved into namespaced message catalogs so that L2 can translate them.

- Create `frontend/messages/en.json` as the source of truth.
- Top-level namespaces: `common` (buttons, labels), `nav`, `home`, `species` (directory + profile), `map`, `dashboard`, `account`, `auth`, `errors`, `glossary`, `about`.
- Walk the App Router pages and components and replace hardcoded English strings with `t("namespace.key")` calls (`useTranslations` in client components, `getTranslations` in server components).
- Pluralization: use ICU MessageFormat for any count-aware strings (the Gate 07 copy-pass notes flagged "one locality record" vs. "N locality records" specifically — get those into `{count, plural, one {...} other {...}}` form).
- Create `frontend/messages/fr.json`, `de.json`, `es.json` as **byte-identical copies of `en.json`**. They're the placeholders L2 / L3 / L5 / L6 fill in.
- Add `npm run i18n:check` script that fails CI if any locale's JSON has different keys from `en.json`. (Custom node script — small.)

Acceptance:

- No hardcoded English strings remain in `frontend/app/` or `frontend/components/` (verified by a `rg` sweep against a banned-pattern list — story includes adding that to CI).
- Three placeholder catalogs exist with identical key sets.
- CI fails if a key is added to `en.json` and not to the others.
- Site renders identically to pre-L1 state for English visitors.

S7 — Locale switcher component (flag-gated)

Audience: All visitors (when `NEXT_PUBLIC_FEATURE_I18N=1`). **Checkpoint:** B. **Scope:** Frontend. **Complexity:** S.

As a visitor whose preferred language is French, German, or Spanish, I want a visible language switcher in the site header so that I can change the site's language.

- New `<LocaleSwitcher />` client component in `frontend/components/`.
- Renders as a small dropdown showing the four language names in their own scripts (English, Français, Deutsch, Español).
- Clicking a language navigates to the equivalent URL on the new locale, preserving the path and query string. Use `next-intl`'s `useRouter()` and `usePathname()`.

- Active locale is visually marked.
- Persists choice in a cookie (`NEXT_LOCALE`) so a return visitor lands on their last-used locale even when they enter via `/` . Cookie has 1-year max-age, `SameSite=Lax` , `Secure` in production.
- Wrapped in a `NEXT_PUBLIC_FEATURE_I18N` check so it doesn't render at all when the flag is off.

Acceptance:

- With flag off: switcher is not in the DOM.
- With flag on: switcher appears in the header. Clicking "Français" while on `/species/123` navigates to `/fr/species/123` .
- Cookie persists. Visiting `/` after selecting French in a previous session redirects to `/fr` (via middleware).
- Keyboard-accessible: focus ring, arrow-key navigation in the dropdown, Enter selects.

S8 — Locale-aware metadata and `hreflang` tags

Audience: Search engines, social sharing. **Checkpoint:** B. **Scope:** Frontend. **Complexity:** S.

As a search engine, I want each page to declare its alternate locales so that I can index the right URL per audience.

- Create a shared `frontend/lib/seo.ts` helper (architect's pick — see architecture doc §2) called from each page's `generateMetadata()` . Helper produces the `alternates.languages` map plus `canonical` URL for the active locale.
- For every page, emit:
  - `<link rel="canonical" href="https://.../<locale-prefix>/path">`
  - `<link rel="alternate" hreflang="en" href="https://.../path">`
  - `<link rel="alternate" hreflang="fr" href="https://.../fr/path">`
  - `<link rel="alternate" hreflang="de" href="https://.../de/path">`
  - `<link rel="alternate" hreflang="es" href="https://.../es/path">`
  - `<link rel="alternate" hreflang="x-default" href="https://.../path">`
- `<title>` , `<meta description>` , `og:title` , `og:description` come from the message catalogs (so they translate when L2 ships).
- **A1 — Sitemap and `robots.txt` (architect addition).** Add `frontend/app/sitemap.ts` (Next.js sitemap convention) emitting all public URLs with cross-locale `xhtml:link` annotations per architecture doc §7. Add `frontend/app/robots.ts` declaring `Sitemap: https://<host>/sitemap.xml` . Lands ahead of any locale flip so search engines see the alternate-URL pattern from day one. ~half-day add.

Acceptance:

- View-source on any page shows all five `hreflang` links plus a `<link rel="canonical">` pointing at the active locale's URL.
- Title / description in `<head>` are catalog-driven, not hardcoded.
- Lighthouse SEO score does not regress.
- `https://<host>/sitemap.xml` returns valid XML with `xhtml:link` alternate annotations for all four locales per public URL.
- `https://<host>/robots.txt` returns a valid robots file referencing the sitemap.

S9 — Fonts widened to `latin-ext`

Audience: Visitors reading German, French, Spanish. **Checkpoint:** B. **Scope:** Frontend. **Complexity:** XS.

As a German reader, I want umlauts (ä, ö, ü, ß) to render in the site's actual typeface rather than a fallback, so that the platform reads as polished.

- Update `frontend/app/layout.tsx` : change `subsets: ["latin"]` to `subsets: ["latin", "latin-ext"]` for Spectral, IBM Plex Sans, and IBM Plex Mono.
- Verify no other custom font loads exist that need the same change.

Acceptance:

- Test page rendering "Müller, Citroën, niño, Aßmann" shows correct glyphs in the site typeface.
- Bundle size impact noted in PR description (latin-ext adds ~10–15KB per font weight).

S10 — Feature flag wiring

Audience: Operators (Vercel env), staff previewers. **Checkpoint:** B. **Scope:** Full-stack (env + frontend logic). **Complexity:** XS.

As an operator, I want a single env var to control whether visitors see any sign of i18n, so that we can ship L1 to production without exposing partial work.

- `NEXT_PUBLIC_FEATURE_I18N` documented in `frontend/.env.example` .
- When the flag is **off**: middleware does not match locale prefixes; `/fr/...` returns 404; `<LocaleSwitcher />` does not render; `hreflang` tags are not emitted.
- When the flag is **on**: full locale routing active.
- Per-locale flags `NEXT_PUBLIC_FEATURE_I18N_FR` , `..._DE` , `..._ES` (default off) further gate which locale prefixes are accepted at the middleware level and which entries appear in the switcher. `NEXT_PUBLIC_FEATURE_I18N=1` plus `..._FR=1` enables only French in production while DE/ES are still in review.
- Document the flag rollout sequence in `docs/planning/i18n/README.md` with a follow-up edit (so future-you knows to flip `..._FR=1` first, then `..._DE` , then `..._ES`).

Acceptance:

- Flag-off: site behaves identically to pre-L1.
- Flag-on with no per-locale flags: switcher visible but only English route active (`/fr/...` 404s).
- Flag-on with `..._FR=1` : French route active; German/Spanish 404. Switcher shows English + Français only.

Integration

S11 — End-to-end smoke test, all four locales

Audience: QA / CI. **Checkpoint:** Full L1. **Scope:** Frontend test suite. **Complexity:** S.

As a maintainer, I want CI to verify every locale loads every key page so that L2's content work doesn't break the framework.

- Playwright (or whatever the project uses for e2e) test that, with the flag on:
  - Loads `/` , `/species/` , `/species/[any id]/` , `/map/` , `/about/` for each locale (`/` , `/fr/` , `/de/` , `/es/`).
  - Asserts `<html lang>` matches the URL prefix.
  - Asserts the `hreflang` tags exist.
  - Asserts the locale switcher is present.
  - For non-English locales, asserts that fallback English content renders (since L1 produces no translations).

- **A4 — Locale-aware caching test (architect addition).** Add a back-to-back-request test: hit `/` with `Accept-Language: fr`, immediately hit `/` with `Accept-Language: en`, assert each gets its expected locale. Catches Vercel cache-key misconfiguration where a French response could leak into an English visitor's render. See architecture doc §1 for the cache-key pattern this verifies.
- Run in CI on every PR.

Acceptance:

- All twenty smoke checks (5 pages × 4 locales) pass green.
- Fails if any page returns a 5xx in any locale.
- The cache-key contamination test (A4) passes — back-to-back requests with different `Accept-Language` headers each render the expected locale.

S12 — Documentation update

Audience: Future maintainers, agents picking up L2+. **Checkpoint:** Full L1. **Scope:** Docs. **Complexity:** XS.

- Update `CLAUDE.md` with a new section: "i18n (Gate L1+)" pointing at `docs/planning/i18n/README.md` and noting:
  - The four supported locales.
  - The flag (`NEXT_PUBLIC_FEATURE_I18N`).
  - Where messages live (`frontend/messages/`).
  - The translation pipeline status model.
  - The "no hardcoded strings" rule going forward.
- Update `frontend/.env.example` and `backend/.env.example` with the new vars (`NEXT_PUBLIC_FEATURE_I18N` , `NEXT_PUBLIC_FEATURE_I18N_FR/DE/ES` , `DEEPL_API_KEY`).

---

2. Out of scope for L1

The following are explicitly **not** L1 work; they are L2/L3:

- Producing any French / German / Spanish content (L2 = UI strings, L3 = species content + admin review UI).
- DeepL client wiring (L3 — no MT calls happen in L1).
- The side-by-side admin review screen (L3).
- Conservation-writer agent invocation in CI or via management command (L3+).
- **B1 — `TranslationStatus` signal handlers** (architect deferral). Auto-creating / auto-invalidating rows on translatable-field saves is L3 work; L1 leaves the model in admin-edit-only mode. See architecture doc §5.
- **B2 — `human_approved` review-gate enforcement** (architect deferral). The `<field>_locale_actual` mixin in S3 checks "column populated/empty" only in L1; the gating against `TranslationStatus.status == human_approved` is L3, controlled by the `I18N_ENFORCE_REVIEW_GATE` setting that flips True in L3.
- Translating Django admin custom labels with `gettext_lazy` (L4 — most of admin gets this for free; we do the custom labels at the point we're translating the coordinator dashboard).
- Translating transactional emails (L4).
- **Localized search across translatable fields** (Q-NEW2, L3+). `SpeciesViewSet.search_fields` only references locale-agnostic columns today; localizing search affects rank stability and is a separate UX call. Architecture doc §12 Q-NEW2.

3. Verification gates

After implementation, before merging to main:

- `@security-reviewer` — review the locale-cookie handling, the per-locale feature-flag enforcement, and confirm no new auth/data-access regressions. Specifically: locale switching must not interact with the DRF token in any way, and the `Accept-Language`-driven response must not leak content above the requesting user's tier.
- `@code-quality-reviewer` — review the App Router restructure, the message-catalog organization, and the modeltranslation registration pattern.
- `@test-writer` — verify the acceptance criteria above against the implementation; write any tests not already covered.

4. Definition of done

- All twelve stories complete with their acceptance criteria green, including the architect's A1–A4 additions and B1–B2 deferrals.
- CI passes including the new smoke tests, the cache-key contamination test (A4), the eight middleware × locale test cases (A3), and the `i18n:check` script.
- Flag is OFF in production (`NEXT_PUBLIC_FEATURE_I18N` not set or `=0`); site behavior identical to pre-L1 for public visitors.
- `gate/L1-i18n-framework` branch merged to main behind the flag.
- `docs/planning/i18n/README.md` Gate L1 row updated to ✅.

5. Implementation order (architect-recommended)

Per architecture doc §8, the dependency-ordered sequence is:

- **Wave 1 (day 1–2, parallel):** S1 (Django locale middleware), S9 (latin-ext fonts), S10 (feature flag wiring). All XS/S complexity, no inter-dependencies, no merge conflicts.
- **Wave 2 (day 2–4, parallel):** S2 (modeltranslation registration), S5 (next-intl + composed middleware + `[locale]` restructure). Backend × frontend parallel; S5 is the critical-path frontend invasion.
- **Wave 3 (day 4–6, parallel):** S3 (DRF locale + `apiFetch` Accept-Language), S4 (TranslationStatus model), S6 (message catalogs — bottleneck, splittable across namespaces), S7 (locale switcher), S8 (hreflang + sitemap + robots.txt). S8 lands after S6's first pass since it consumes catalog keys.
- **Wave 4 (day 6–7, sequential):** S11 (e2e smoke tests including A4 cache-key test), S12 (docs).

Total estimated effort: ~8–8.5 working days. Target ship: May 8 (with ~1 week slack from a May 1 start).